

```
# MAYCON APAZA CONDORI
#Importación de librerías
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Importación del dataset con separador correcto
dataset = pd.read_csv('social_network_ads.csv', sep=';')
dataset.info() # Información del dataset
dataset.head(5) # Mostrar las primeras 5 filas

# X son los datos de edad y salario estimado
X = dataset.iloc[:, [2, 3]].values
# y es la decisión de compra (1 si compra, 0 si no compra)
y = dataset.iloc[:, 4].values

# División en datos de entrenamiento y prueba
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=4)

# Normalización o ajuste de escalas
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
print('Datos de X_train y X_test:')
print(X_train, X_test)

# Ajuste de regresión logística al conjunto de entrenamiento
from sklearn.linear_model import LogisticRegression
classifier = LogisticRegression(random_state=0)
classifier.fit(X_train, y_train)

# Predicción del conjunto de pruebas
y_pred = classifier.predict(X_test)
print('Predicción de valores para X_test:')
print(y_pred)

# La precisión del modelo se obtiene con el score
score_test = classifier.score(X_test, y_test)
print('Precisión del modelo:', score_test)

# Matriz de Confusión
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print('Matriz de confusión:')
print(cm)
```

```

[-0.28215842  0.07433031]
[ 1.3472915  1.25070378]
[-1.04895838 -0.38474227]
[ 0.96389151 -1.07335113]
[ 0.29294155  0.24648252]
[-1.24065838  0.56209492]
[ 1.73069148  0.96378342]
[-0.85725839  2.226233 ]
[ 0.86804152 -0.55689448]
[ 0.29294155 -0.52820245]
[-0.6655584  0.53340288]
[-1.71990835  0.33255863]
[-0.6655584  0.10302234]
[-0.18630843 -1.07335113]
[-0.95310839  0.53340288]
[-1.43235837 -0.64297059]
[ 1.05974151  0.10302234]
[-1.71990835  0.10302234]
[ 0.86804152 -0.61427856]
[-0.47385841 -0.78643077]
[-1.81575835 -1.47503963]
[-0.85725839  0.36125066]
[ 0.38879154 -0.15520598]
[-0.28215842 -0.38474227]

```

```

# =====
#   REGRESIÓN LOGÍSTICA - CHURN DATASET
# =====

# Importar las librerías
import pandas as pd
import pylab as pl
import numpy as np
import scipy.optimize as opt
from sklearn import preprocessing
import matplotlib.pyplot as plt

# -----
# Datos de una compañía de telecomunicaciones que
# busca retener a sus clientes. Cada caso representa
# un cliente, con información demográfica y de uso
# del servicio.
# -----

# Descargar o cargar el archivo ChurnData.csv
# Si ya lo tienes descargado, súbelo a Colab y usa: pd.read_csv("ChurnData.csv")
# Si quieres cargarlo directamente desde el enlace original:
url = "https://s3-api.us-gio.objectstorage.softlayer.net/cf-courses-data/CognitiveClass/ML0101ENV3/labs/ChurnData.csv"
churn_df = pd.read_csv(url)

# Mostrar información del dataset
churn_df.info()
print("\nÚltimas 5 filas del dataset:")
print(churn_df.tail(5))

# Seleccionar características relevantes y convertir el target a entero
churn_df = churn_df[['tenure', 'age', 'address', 'income',
                    'ed', 'employ', 'equip', 'callcard',
                    'wireless', 'churn']]
churn_df['churn'] = churn_df['churn'].astype('int')

print('\nVariables (columnas) del archivo churn:')
print(churn_df.head())

# Definir X (variables independientes) y y (variable dependiente)
X = np.asarray(churn_df[['tenure', 'age', 'address', 'income',
                        'ed', 'employ', 'equip']])
print('\nSet de datos X (primeras 5 filas):')
print(X[0:5])

y = np.asarray(churn_df['churn'])
print('\nSet de datos y (primeras 5 etiquetas):')
print(y[0:5])

# Normalizar el set de datos
X = preprocessing.StandardScaler().fit(X).transform(X)
print("\nDatos normalizados correctamente.")

# =====
#   EJEMPLO 2 - División de datos y entrenamiento
# =====

# Dividir el set de datos en entrenamiento y prueba
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=4)

```

```

print('\nTrain set:', X_train.shape, y_train.shape)
print('Test set:', X_test.shape, y_test.shape)

# Entrenar el modelo de regresión logística
from sklearn.linear_model import LogisticRegression
LR = LogisticRegression(C=0.01, solver='liblinear').fit(X_train, y_train)
print('\nModelo de regresión logística entrenado:')
print(LR)

# Predicción con el set de prueba
yhat = LR.predict(X_test)
print('\nPredicción con el set de prueba:')
print(yhat)

# =====
# Evaluación del modelo
# =====

from sklearn.metrics import classification_report, confusion_matrix
cnf_matrix = confusion_matrix(y_test, yhat, labels=[1, 0])
print('\nMatriz de confusión:')
print(cnf_matrix)

# Reporte de métricas: precisión, recall y F1-score
print('\nReporte de clasificación:')
print(classification_report(y_test, yhat))

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 28 columns):

```

| #  | Column   | Non-Null Count | Dtype   |
|----|----------|----------------|---------|
| 0  | tenure   | 200 non-null   | float64 |
| 1  | age      | 200 non-null   | float64 |
| 2  | address  | 200 non-null   | float64 |
| 3  | income   | 200 non-null   | float64 |
| 4  | ed       | 200 non-null   | float64 |
| 5  | employ   | 200 non-null   | float64 |
| 6  | equip    | 200 non-null   | float64 |
| 7  | callcard | 200 non-null   | float64 |
| 8  | wireless | 200 non-null   | float64 |
| 9  | longmon  | 200 non-null   | float64 |
| 10 | tollmon  | 200 non-null   | float64 |
| 11 | equipmon | 200 non-null   | float64 |
| 12 | cardmon  | 200 non-null   | float64 |
| 13 | wiremon  | 200 non-null   | float64 |
| 14 | longten  | 200 non-null   | float64 |
| 15 | tollten  | 200 non-null   | float64 |
| 16 | cardten  | 200 non-null   | float64 |
| 17 | voice    | 200 non-null   | float64 |
| 18 | pager    | 200 non-null   | float64 |
| 19 | internet | 200 non-null   | float64 |
| 20 | callwait | 200 non-null   | float64 |
| 21 | confer   | 200 non-null   | float64 |
| 22 | ebill    | 200 non-null   | float64 |
| 23 | loglong  | 200 non-null   | float64 |
| 24 | logtoll  | 200 non-null   | float64 |
| 25 | lninc    | 200 non-null   | float64 |
| 26 | custcat  | 200 non-null   | float64 |
| 27 | churn    | 200 non-null   | float64 |

dtypes: float64(28)

memory usage: 43.9 KB

Últimas 5 filas del dataset:

|     | tenure | age  | address | income | ed  | employ | equip | callcard | wireless | \ |
|-----|--------|------|---------|--------|-----|--------|-------|----------|----------|---|
| 195 | 55.0   | 44.0 | 24.0    | 83.0   | 1.0 | 23.0   | 0.0   | 1.0      | 0.0      |   |
| 196 | 34.0   | 23.0 | 3.0     | 24.0   | 1.0 | 7.0    | 0.0   | 1.0      | 0.0      |   |
| 197 | 6.0    | 32.0 | 10.0    | 47.0   | 1.0 | 10.0   | 0.0   | 1.0      | 0.0      |   |
| 198 | 24.0   | 30.0 | 0.0     | 25.0   | 4.0 | 5.0    | 0.0   | 1.0      | 1.0      |   |
| 199 | 61.0   | 50.0 | 16.0    | 190.0  | 2.0 | 22.0   | 1.0   | 1.0      | 1.0      |   |

|     | longmon | ... | pager | internet | callwait | confer | ebill | loglong | logtoll | \ |
|-----|---------|-----|-------|----------|----------|--------|-------|---------|---------|---|
| 195 | 17.35   | ... | 0.0   | 0.0      | 0.0      | 1.0    | 0.0   | 2.854   | 3.199   |   |
| 196 | 6.00    | ... | 0.0   | 0.0      | 1.0      | 1.0    | 0.0   | 1.792   | 3.332   |   |
| 197 | 3.85    | ... | 0.0   | 0.0      | 1.0      | 1.0    | 0.0   | 1.348   | 3.168   |   |
| 198 | 8.70    | ... | 1.0   | 1.0      | 1.0      | 1.0    | 1.0   | 2.163   | 3.866   |   |
| 199 | 16.85   | ... | 0.0   | 1.0      | 0.0      | 0.0    | 1.0   | 2.824   | 3.240   |   |

|     | lninc | custcat | churn |
|-----|-------|---------|-------|
| 195 | 4.419 | 3.0     | 0.0   |
| 196 | 3.178 | 3.0     | 0.0   |
| 197 | 3.850 | 3.0     | 0.0   |
| 198 | 3.219 | 4.0     | 1.0   |
| 199 | 5.247 | 2.0     | 0.0   |

Empieza a programar o a [crear código](#) con IA.

```

# MAYCON APAZA CONDORI
# =====
# PROCESAMIENTO DE DATOS - ESTUDIANTES
# =====

import pandas as pd

print("Cargando datos de Estudiantes...")

# Cargar archivo estudiantes.csv (usa separador ;)
df = pd.read_csv("estudiantes.csv", sep=";")

# Mostrar información general del dataset
df.info()

# Eliminar datos nulos
df.dropna(inplace=True)

# Mostrar información después de eliminar los nulos
df.info()

# Mostrar información estadística de todos los campos numéricos
print("\n=== DESCRIPCIÓN GENERAL ===")
print(df.describe())

# Mostrar información solo del campo "Edad"
print("\n=== CAMPO: Edad ===")
print(df.describe()[["Edad"]])

# Mostrar información de dos columnas: Ingresos y Gastos
print("\n=== CAMPOS: Ingresos y Gastos ===")
print(df.describe()[["Ingresos", "Gastos"]])

# Número de valores únicos del campo Sexo
print("\nValores únicos en 'Sexo':", df["Sexo"].nunique())

# Generar una Serie a partir de una columna
df2 = df["Sexo"]
print("\nSerie df2 (Sexo):")
print(df2)

# Generar un DataFrame a partir de una columna
df3 = pd.DataFrame(df["Sexo"])
print("\nDataFrame df3:")
print(df3)

# Crear columnas Hombre y Mujer a partir de la columna Sexo
df3["Hombre"] = df3["Sexo"].apply(lambda x: 1 if x == "Hombre" else 0)
df3["Mujer"] = df3["Sexo"].apply(lambda x: 1 if x == "Mujer" else 0)

# Eliminar la columna original Sexo
df3.drop(["Sexo"], axis=1, inplace=True)

print("\nDataFrame df3 con columnas binarias (Hombre/Mujer):")
print(df3)

# Concatenar df original con df3
dff = pd.concat([df, df3], axis=1)
print("\nDataFrame final concatenado:")
print(dff)

```

```

Cargando datos de Estudiantes...
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1300 entries, 0 to 1299
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype
---  -
0   ID           1300 non-null   int64
1   Edad         1299 non-null   float64
2   Depto        1298 non-null   object
3   Sexo         1299 non-null   object
4   Reside       1300 non-null   object
5   NroMat       1300 non-null   int64
6   PromNotas    1299 non-null   float64
7   Ingresos     1299 non-null   float64
8   Gastos       1299 non-null   float64
9   Internet     1300 non-null   object
dtypes: float64(4), int64(2), object(4)
memory usage: 101.7+ KB
<class 'pandas.core.frame.DataFrame'>
Index: 1293 entries, 0 to 1299

```

```
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype
---  -
0   ID           1293 non-null    int64
1   Edad         1293 non-null    float64
2   Depto        1293 non-null    object
3   Sexo         1293 non-null    object
4   Reside       1293 non-null    object
5   NroMat       1293 non-null    int64
6   PromNotas    1293 non-null    float64
7   Ingresos     1293 non-null    float64
8   Gastos       1293 non-null    float64
9   Internet     1293 non-null    object
dtypes: float64(4), int64(2), object(4)
memory usage: 111.1+ KB
```

```
=== DESCRIPCIÓN GENERAL ===
      ID      Edad      NroMat      PromNotas      Ingresos  \
count  1293.000000  1293.000000  1293.000000  1293.000000  1293.000000
mean   1649.798144   23.423821   25.475638   71.282289  1129.683681
std     375.621244    3.634176    8.344112   10.869332   221.748602
min     1001.000000   19.000000   12.000000   51.000000   750.000000
25%    1325.000000   21.000000   18.000000   63.000000   933.000000
50%    1648.000000   23.000000   26.000000   71.000000  1141.000000
75%    1975.000000   26.000000   33.000000   79.000000  1321.000000
max     2300.000000   86.000000   39.000000  130.000000  1499.000000

      Gastos
count  1293.000000
mean    808.532869
std     196.931149
min     416.000000
25%     660.000000
50%     790.000000
75%     944.000000
max    1318.000000
```

```
=== CAMPO: Edad ===
```

```
# =====
# PROCESAMIENTO DE DATOS - CLIENTES
# =====

import pandas as pd

print("Cargando datos de Clientes...")

# Cargar archivo clientes.csv con codificación UTF-8
dc = pd.read_csv("clientes.csv", sep=";", encoding="utf8")

# Vista general del archivo
print("\nVista general del archivo clientes.csv:")
print(dc.head())

# Ejemplo de separación de la columna NOMBRES
print("\nSeparación preliminar de la columna NOMBRES:")
pd.DataFrame(dc["NOMBRES"].str.split())

# Separar la columna NOMBRES en tres columnas: Paterno, Materno, Nombres
dc[["Paterno", "Materno", "Nombres"]] = dc["NOMBRES"].str.split(' ', n=2, expand=True)

# Eliminar la columna original NOMBRES
dc.drop(['NOMBRES'], axis=1, inplace=True)

print("\nDataset con columnas separadas:")
print(dc.head())

# Filtrar los apellidos paternos iguales a QUISPE
print("\nClientes con Paterno == 'QUISPE':")
print(dc[dc.Paterno == "QUISPE"])

# Aplicar operadores lógicos (OR, AND)
print("\nClientes con Paterno == 'QUISPE' o 'MAMANI':")
print(dc[(dc.Paterno == "QUISPE") | (dc.Paterno == "MAMANI")])

print("\nClientes con Paterno == 'QUISPE' y Materno == 'MAMANI':")
print(dc[(dc.Paterno == "QUISPE") & (dc.Materno == "MAMANI")])
```

```
Cargando datos de Clientes...
```

```
Vista general del archivo clientes.csv:
      CARNET      NOMBRES
0  15853389  AGNE QUISPE EDDY RICHARD
1  12214535  AGUILAR MAIDANA PAOLA
2  17134405  ALANOCA RAMIREZ SONIA ESTHER
```

```
3 17294817    ANCARI VILLCARANI LICETT
4 12176514    APAZA QUISPE JORGE RONY
```

Separación preliminar de la columna NOMBRES:

Dataset con columnas separadas:

|   | CARNET   | Paterno | Materno    | Nombres      |
|---|----------|---------|------------|--------------|
| 0 | 15853389 | AGNE    | QUISPE     | EDDY RICHARD |
| 1 | 12214535 | AGUILAR | MAIDANA    | PAOLA        |
| 2 | 17134405 | ALANOCA | RAMIREZ    | SONIA ESTHER |
| 3 | 17294817 | ANCARI  | VILLCARANI | LICETT       |
| 4 | 12176514 | APAZA   | QUISPE     | JORGE RONY   |

Clientes con Paterno == 'QUISPE':

|    | CARNET   | Paterno | Materno  | Nombres      |
|----|----------|---------|----------|--------------|
| 59 | 16141060 | QUISPE  | AJNO     | ERICK        |
| 60 | 19111682 | QUISPE  | MAMANI   | ALVARO       |
| 61 | 11875779 | QUISPE  | QUISBERT | ELIZABETH    |
| 62 | 13438129 | QUISPE  | ROQUE    | LIDIA JUDITH |

Clientes con Paterno == 'QUISPE' o 'MAMANI':

|    | CARNET   | Paterno | Materno   | Nombres       |
|----|----------|---------|-----------|---------------|
| 45 | 12194664 | MAMANI  | CHOQUE    | DIEGO ARMANDO |
| 46 | 21172544 | MAMANI  | POMA      | NANCY         |
| 47 | 15557588 | MAMANI  | RODRIGUEZ | DAMIAN        |
| 48 | 14966704 | MAMANI  | UTURUNCO  | MIGUEL        |
| 59 | 16141060 | QUISPE  | AJNO      | ERICK         |
| 60 | 19111682 | QUISPE  | MAMANI    | ALVARO        |
| 61 | 11875779 | QUISPE  | QUISBERT  | ELIZABETH     |
| 62 | 13438129 | QUISPE  | ROQUE     | LIDIA JUDITH  |

Clientes con Paterno == 'QUISPE' y Materno == 'MAMANI':

|    | CARNET   | Paterno | Materno | Nombres |
|----|----------|---------|---------|---------|
| 60 | 19111682 | QUISPE  | MAMANI  | ALVARO  |

Empieza a programar o a [crear código](#) con IA.

```

#MAYCON APAZA CONDORI
#=====
# PREPROCESAMIENTO DE DATOS CON PANDAS (PDF DATOS11)
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 1 Cargar el archivo
df = pd.read_csv("festudiantes.csv", sep=";")
print("Archivo cargado correctamente")
df.info()

# 2 Eliminar registros con 3 o más campos nulos
a = df.isnull().sum(axis=1)
print("\nRegistros con 3 o más campos nulos:")
print(a[a > 2])
indices = [83, 93, 122, 167, 518, 623, 958, 996, 1047, 1177] # solo ejemplo
df1 = df.dropna(thresh=df.shape[1] - 2) # elimina filas con más de 2 nulos
df1.reset_index(drop=True, inplace=True)
print("\nDespués de eliminar nulos:")
df1.info()

# 3 Reemplazar valores nulos por el siguiente registro
df2 = df1.fillna(method="bfill")
print("\nDatos después de reemplazar nulos con siguiente registro:")
df2.info()

# Función para reemplazar valores fuera de rango por la media
def cambia(dfx, columnlist):
    for col in columnlist:
        media = dfx[col].mean()
        desvia = dfx[col].std()
        bajo = int(media - 2 * desvia)
        alto = int(media + 2 * desvia)
        print(f"{col}: Inferior={bajo}, Superior={alto}")
        dfx.loc[dfx[col] < bajo, col] = media
        dfx.loc[dfx[col] > alto, col] = media
    return dfx

dfy = cambia(df2, ["Edad", "HrsPrmEstudio", "PromNotas", "NroMat", "Ingresos", "Gastos"])

# 4 Gráficos con Matplotlib
dfw = df.groupby("Ciudad")["ID"].count().reset_index()
xciudad = [v for v in dfw['Ciudad']]
yid = [v for v in dfw['ID']]
plt.figure(figsize=(12,5))
plt.bar(xciudad, yid)
plt.title("DIAGRAMA DE BARRAS: CIUDAD VS Nº DE ESTUDIANTES")
plt.show()

dfw = dfy.groupby("Edad")["PromNotas"].mean().reset_index()
plt.figure(figsize=(12,5))
plt.plot(dfw["Edad"], dfw["PromNotas"])
plt.title("CURVA PROMEDIO DE NOTAS POR EDAD")
plt.show()

plt.figure(figsize=(10,5))
plt.boxplot(dfy["Edad"])
plt.title("BOXPLOT: EDAD DE ESTUDIANTES")
plt.show()

plt.figure(figsize=(10,5))
plt.hist(df["HrsPrmEstudio"], bins=4, color='skyblue', edgecolor='black')
plt.title("HISTOGRAMA HORAS PROMEDIO DE ESTUDIO")
plt.xlabel("Horas promedio")
plt.ylabel("Frecuencia")
plt.show()

# 5 Gráficos con Seaborn
plt.figure(figsize=(10,5))
sns.histplot(df["HrsPrmEstudio"], kde=False)
plt.title("HISTOGRAMA SEABORN: HORAS PROMEDIO DE ESTUDIO")
plt.show()

plt.figure(figsize=(10,5))
sns.kdeplot(dfy["Edad"], fill=True)
plt.title("CURVA SEABORN: EDAD")
plt.show()

```



```
plt.show()
```

```
plt.figure(figsize=(10,5))
sns.boxplot(x=dfy["Edad"])
plt.title("BOXPLOT SEABORN: EDAD")
plt.show()
```

```
# 6 Reemplazar nulos con la media (función final)
```

```
def fill_missing_values(df):
    for col in df.select_dtypes(include=["int", "float"]).columns:
        val = df[col].mean()
        df[col].fillna(val, inplace=True)
    return df
```

```
df3 = fill_missing_values(df1)
print("\nDataFrame final sin nulos:")
df3.info()
```



```

Archivo cargado correctamente
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1300 entries, 0 to 1299
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ID                     1300 non-null  int64
1   Edad                  1292 non-null  float64
2   Ciudad                1290 non-null  object
3   La_Paz                1300 non-null  int64
4   Cochabamba            1300 non-null  int64
5   Santa_Cruz            1300 non-null  int64
6   Resto                  1300 non-null  int64
7   Hombre                1300 non-null  int64
8   Mujer                 1300 non-null  int64
9   TieneEquipo           1290 non-null  object
10  HrsPrmEstudio          1300 non-null  int64
11  NroMat                 1290 non-null  float64
12  PromNotas              1289 non-null  float64
13  Ingresos               1289 non-null  float64
14  Gastos                 1289 non-null  float64
15  Internet               1300 non-null  object
dtypes: float64(5), int64(8), object(3)
memory usage: 162.6+ KB

```

Registros con 3 o más campos nulos:

```

83      7
93      6
122     7
167     7
518     7
623     7
958     6
996     6
1047    7
1177    7
dtype: int64

```

Después de eliminar nulos:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1290 entries, 0 to 1289
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ID                     1290 non-null  int64
1   Edad                  1289 non-null  float64
2   Ciudad                1290 non-null  object
3   La_Paz                1290 non-null  int64
4   Cochabamba            1290 non-null  int64
5   Santa_Cruz            1290 non-null  int64
6   Resto                  1290 non-null  int64
7   Hombre                1290 non-null  int64
8   Mujer                 1290 non-null  int64
9   TieneEquipo           1290 non-null  object
10  HrsPrmEstudio          1290 non-null  int64
11  NroMat                 1290 non-null  float64
12  PromNotas              1289 non-null  float64
13  Ingresos               1289 non-null  float64
14  Gastos                 1289 non-null  float64
15  Internet               1290 non-null  object
dtypes: float64(5), int64(8), object(3)
memory usage: 161.4+ KB

```

Datos después de reemplazar nulos con siguiente registro:

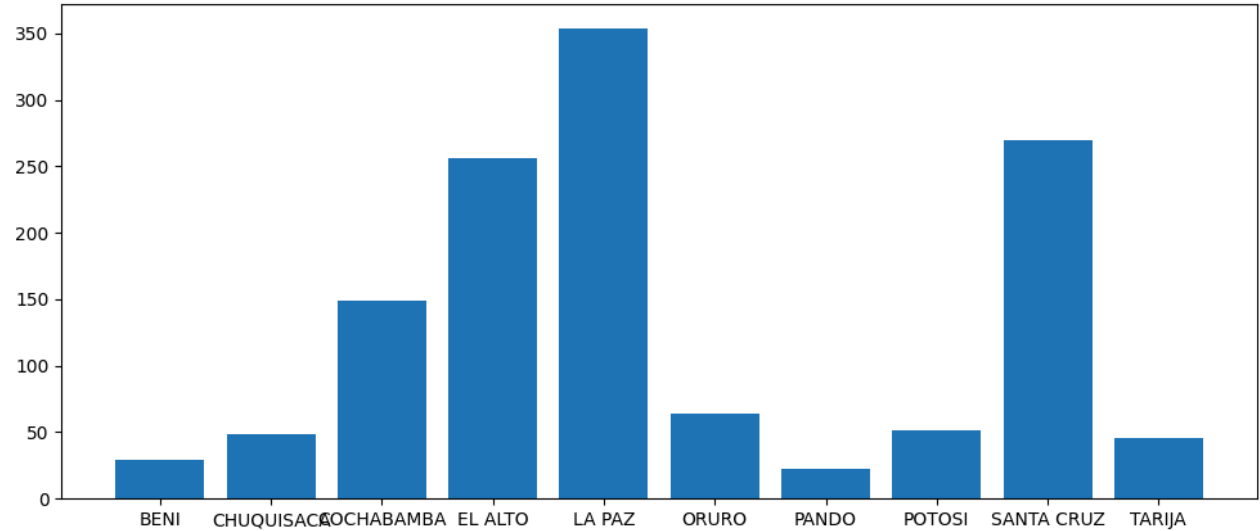
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1290 entries, 0 to 1289
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ID                     1290 non-null  int64
1   Edad                  1290 non-null  float64
2   Ciudad                1290 non-null  object
3   La_Paz                1290 non-null  int64
4   Cochabamba            1290 non-null  int64
5   Santa_Cruz            1290 non-null  int64
6   Resto                  1290 non-null  int64
7   Hombre                1290 non-null  int64
8   Mujer                 1290 non-null  int64
9   TieneEquipo           1290 non-null  object
10  HrsPrmEstudio          1290 non-null  int64
11  NroMat                 1290 non-null  float64
12  PromNotas              1290 non-null  float64
13  Ingresos               1290 non-null  float64
14  Gastos                 1290 non-null  float64
15  Internet               1290 non-null  object
dtypes: float64(5), int64(8), object(3)
memory usage: 161.4+ KB
Edad: Inferior=16, Superior=30
HrsPrmEstudio: Inferior=64, Superior=123
PromNotas: Inferior=49, Superior=92

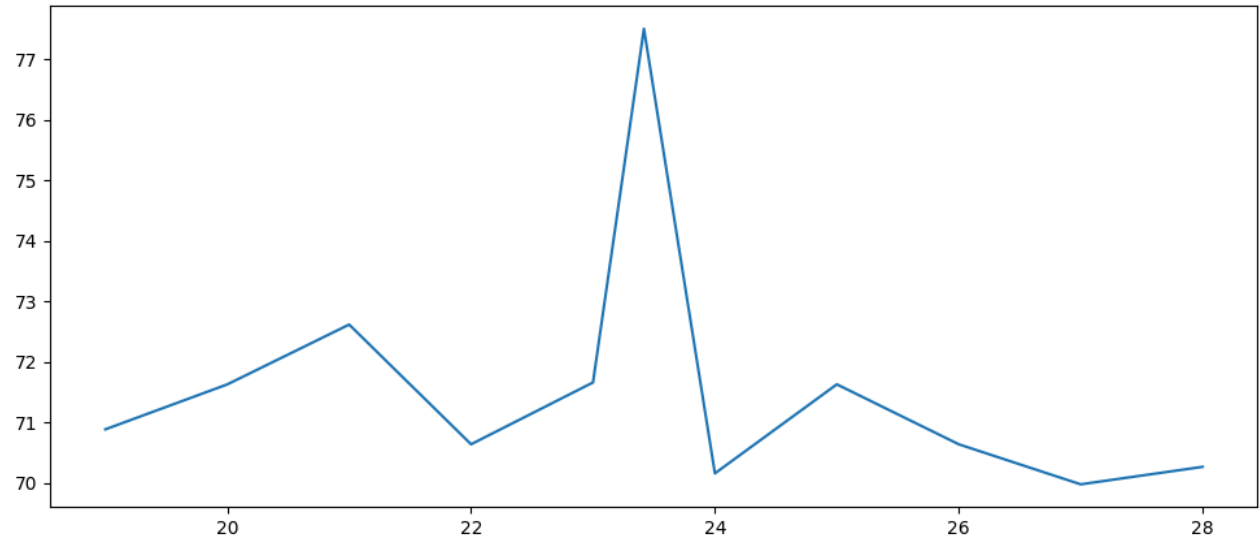
```

```
NroMat: Inferior=8, Superior=42
Ingresos: Inferior=685, Superior=1572
Gastos: Inferior=414, Superior=1201
/tmp/ipython-input-2273116978.py:26: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future
df2 = df1.fillna(method="bfill")
/tmp/ipython-input-2273116978.py:38: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an er
dfx.loc[dfx[col] < bajo, col] = media
```

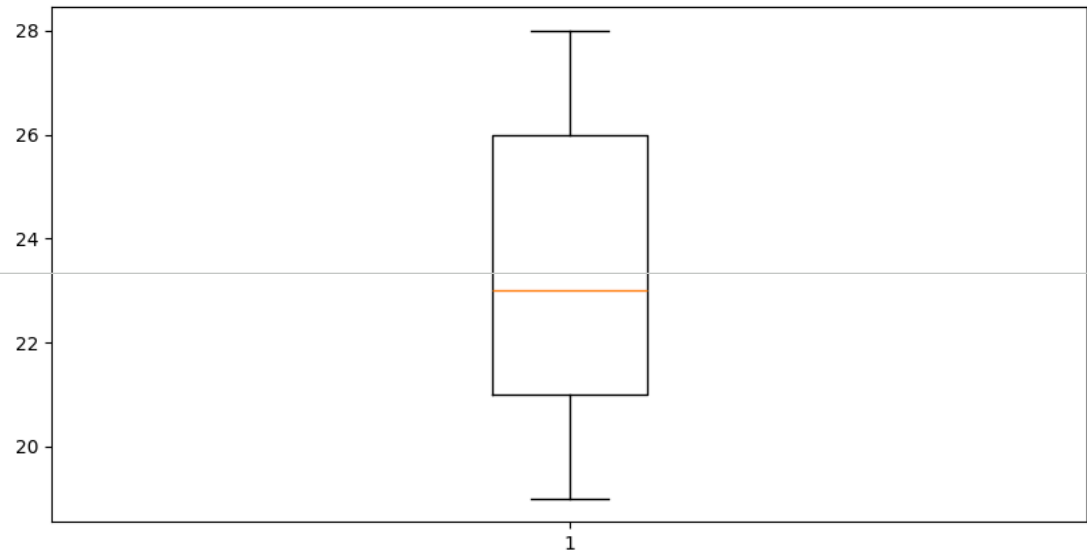
DIAGRAMA DE BARRAS: CIUDAD VS Nº DE ESTUDIANTES



CURVA PROMEDIO DE NOTAS POR EDAD



BOXPLOT: EDAD DE ESTUDIANTES



HISTOGRAMA HORAS PROMEDIO DE ESTUDIO

